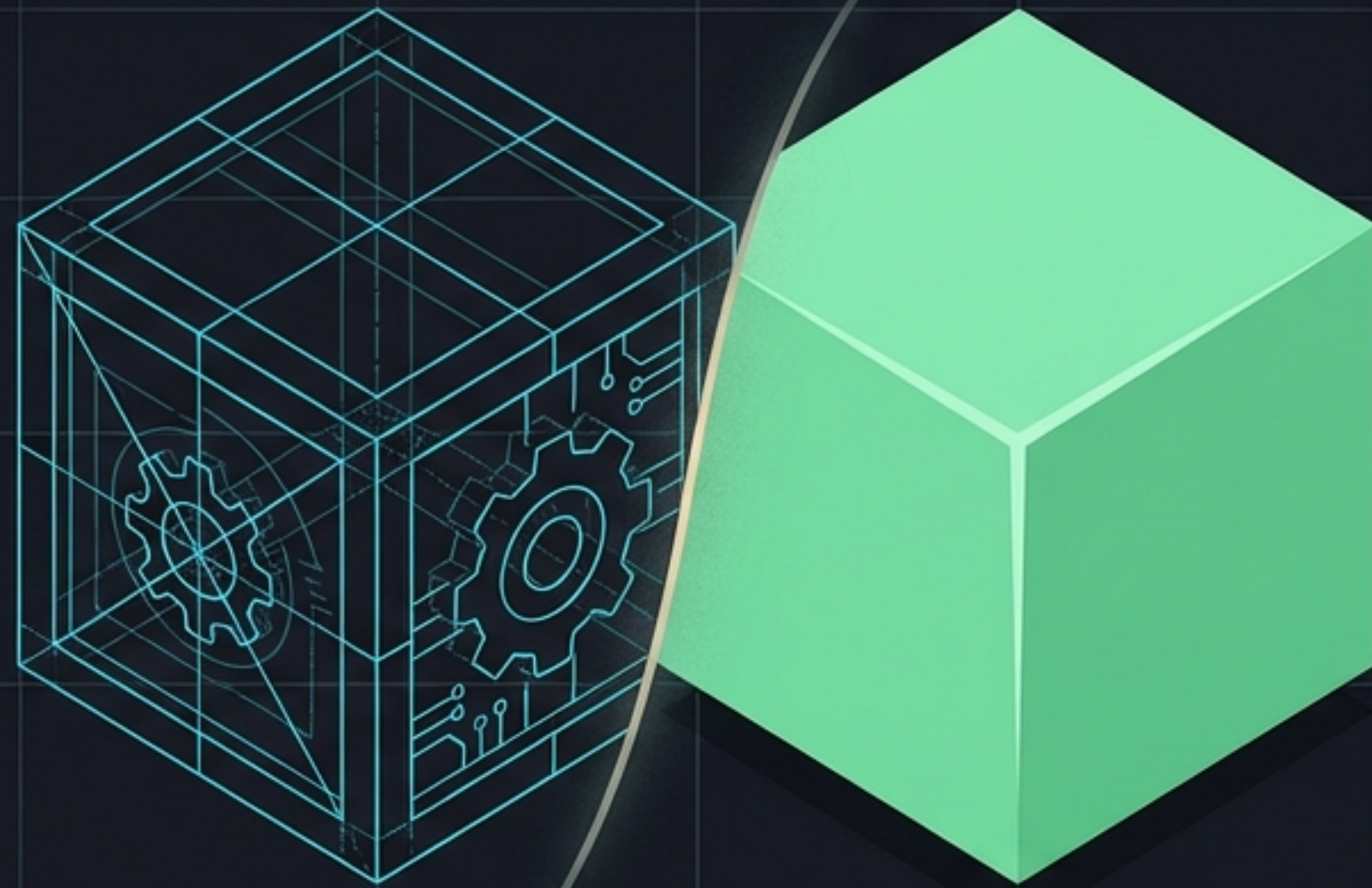
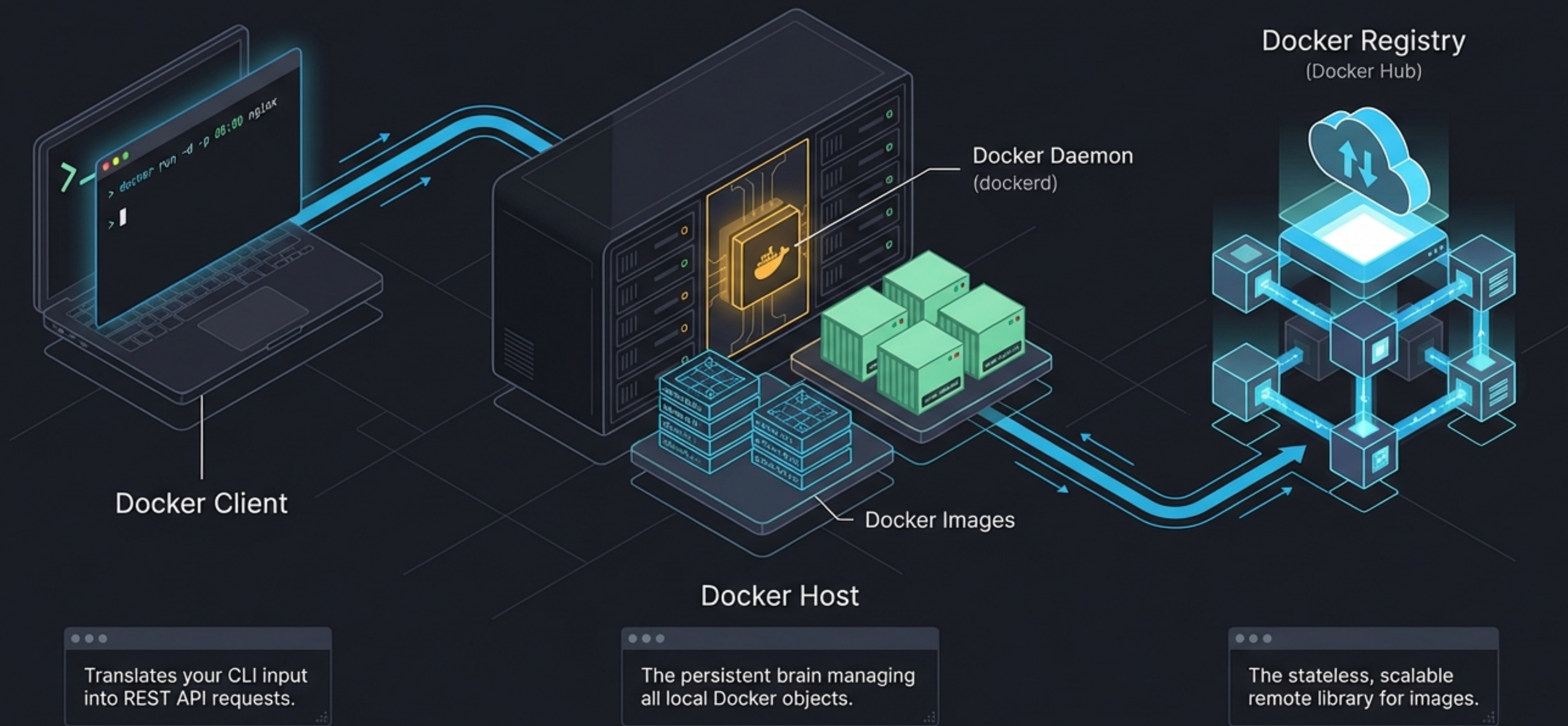


Mastering the Docker CLI



The Object-Oriented Grammar of Container Management

DOCKER ARCHITECTURE: THE ISOMETRIC DATA FLOW

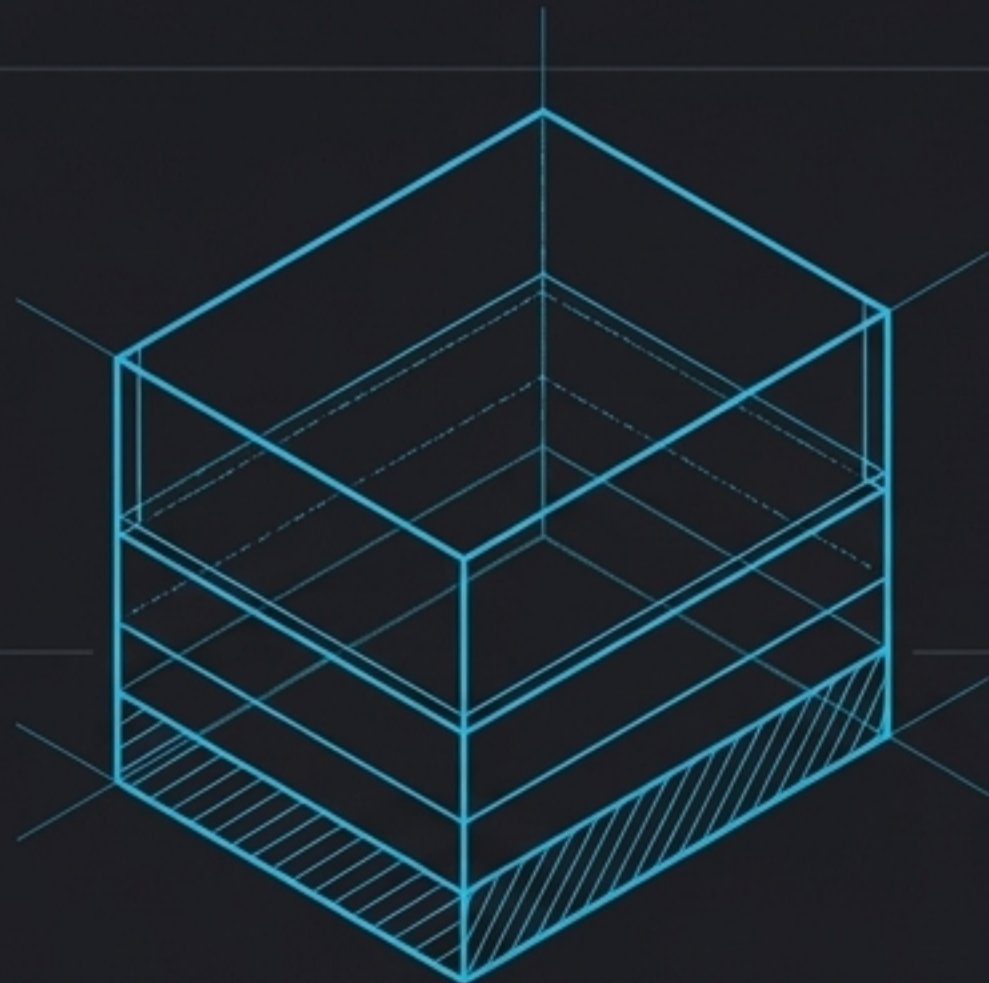


The CLI Evolution

Legacy Syntax	Modern Syntax	Docker Object	Purpose
<code>docker ps</code>	<code>docker container ls</code>	 <code>container</code>	List active instances
<code>docker run</code>	<code>docker container run</code>	 <code>container</code>	Run command in new container
<code>docker rm</code>	<code>docker container rm</code>	 <code>container</code>	Remove stopped instances
<code>docker images</code>	<code>docker image ls</code>	 <code>image</code>	List stored blueprints
<code>docker rmi</code>	<code>docker image rm</code>	 <code>image</code>	Remove stored blueprints

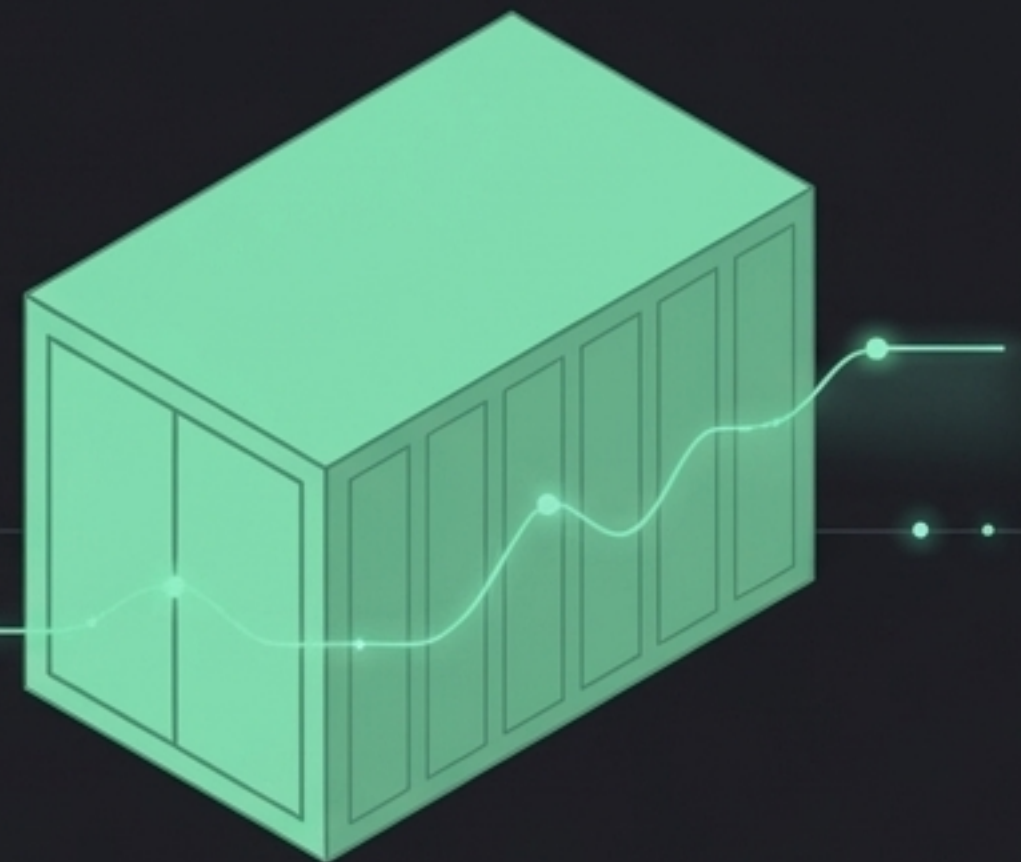
Modern Docker commands follow a strict grammar: `docker <object> <verb>`

Docker Image



Containers are instantiated from Images.

Docker Container



Read-Only

Inert Template

Built in Layers

Contains application code and dependencies

Runnable

Live Instance

Ephemeral Writable Layer

Isolated process space

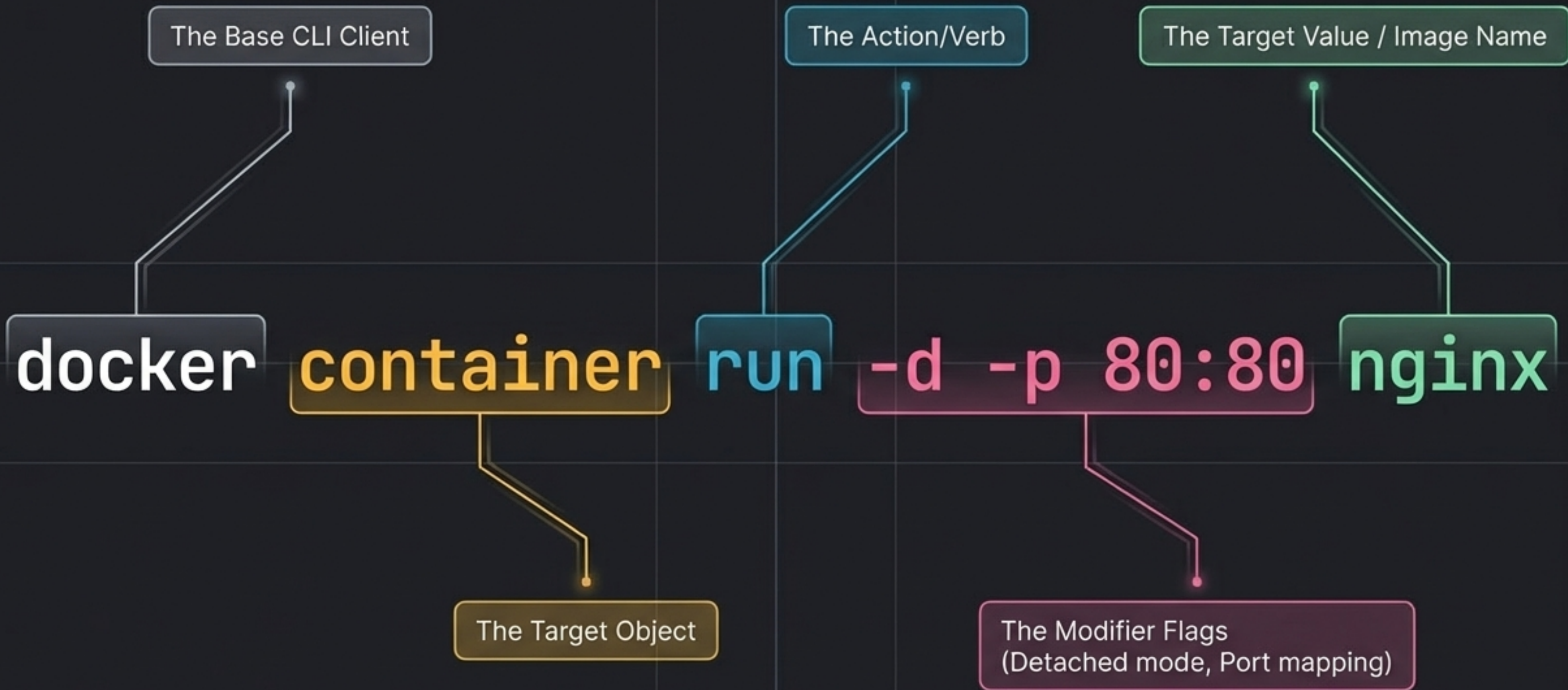


Image Management: Acquisition

```
docker login
```

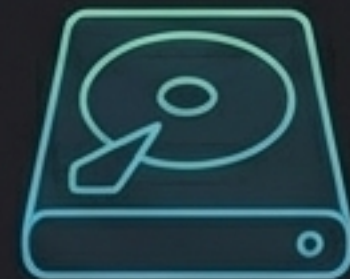
Authenticates your local daemon with Docker Hub to push/pull private images.

```
docker pull <image_name>:<tag>
```

Downloads the blueprint to your local daemon. Defaults to the **:latest** tag if none is specified.



Registry



Host

Image Management: Creation & Sharing

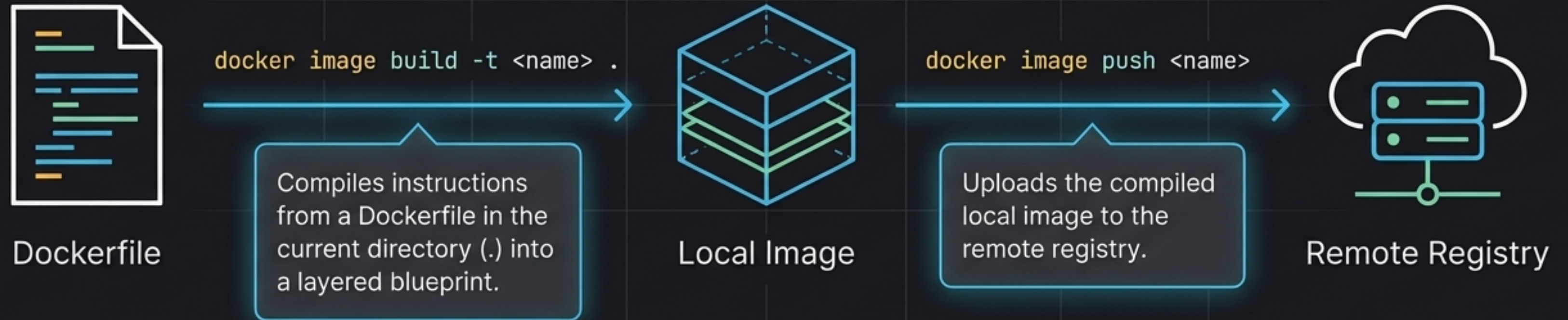


Image Management: Audit & Cleanup

Audit

```
docker image ls
```

(Legacy: `docker images`)

REPOSITORY	TAG	IMAGE ID	SIZE
nginx	latest	a4b5c6d7	133MB
ubuntu	20.04	f1g2h3j4	72.5MB

Lists all pulled and built images currently stored on the local daemon.

Cleanup

```
docker image rm <image_id>
```

(Legacy: `docker rmi`)



Deletes the inert image template. Frees up disk space from useless or outdated layers.

docker container run

- -name

Assigns a human-readable name instead of a random hash.

- d

Detached mode. Runs the container in the background, freeing up the terminal.

$-i$

Interactive. Keeps STDIN open even if not attached

- e

Environment variables.
Injects config settings into
the live container.



If the image is not found locally, 'run' automatically triggers a 'pull' first.

Container Management: Monitoring



```
docker container ls
```

(Legacy: `docker ps`)

Shows only the currently running containers.

```
docker container ls -a
```

The `-a` flag reveals ALL containers, whether running, stopped, or crashed.

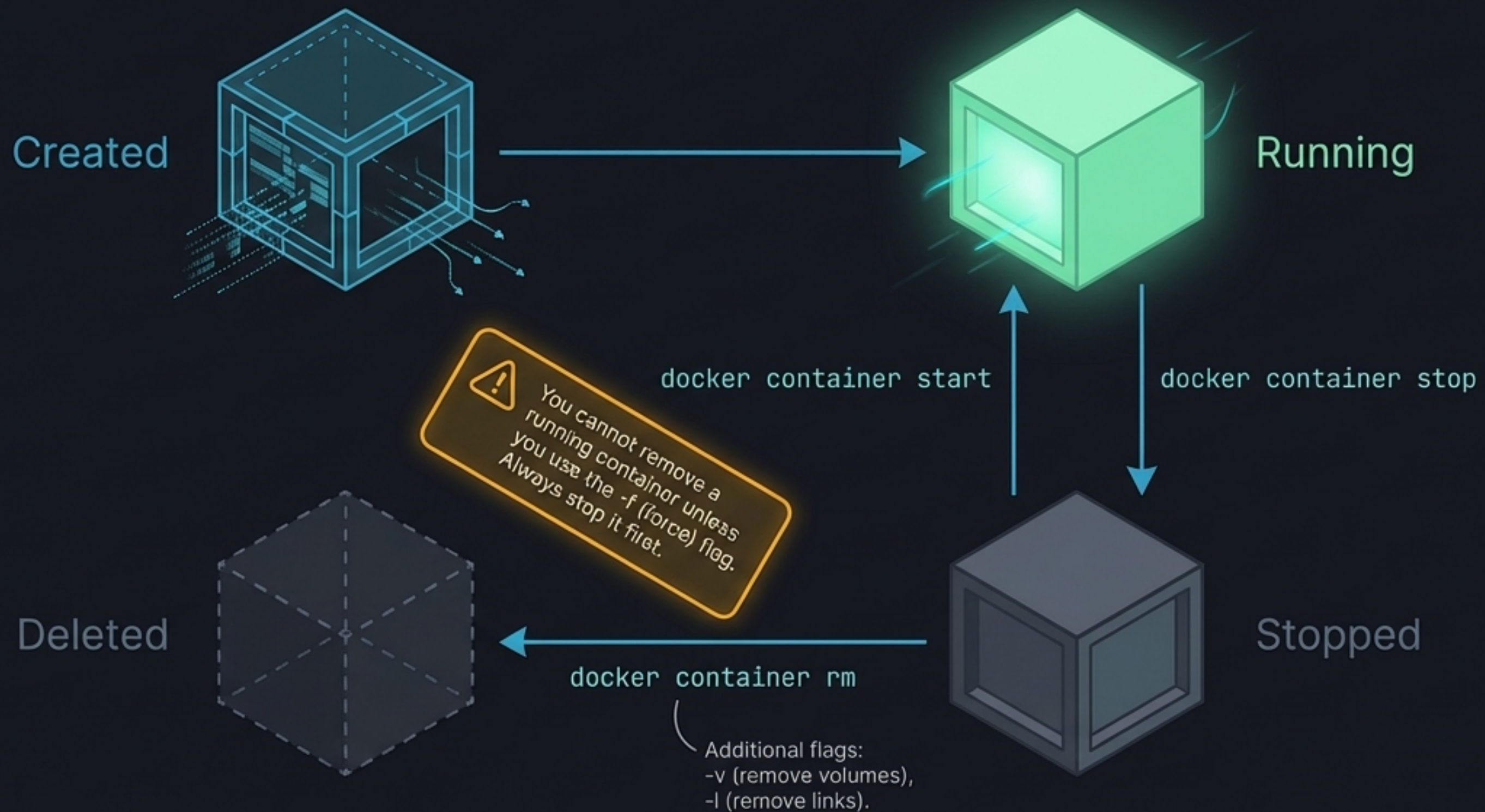
```
-l
```

shows latest container

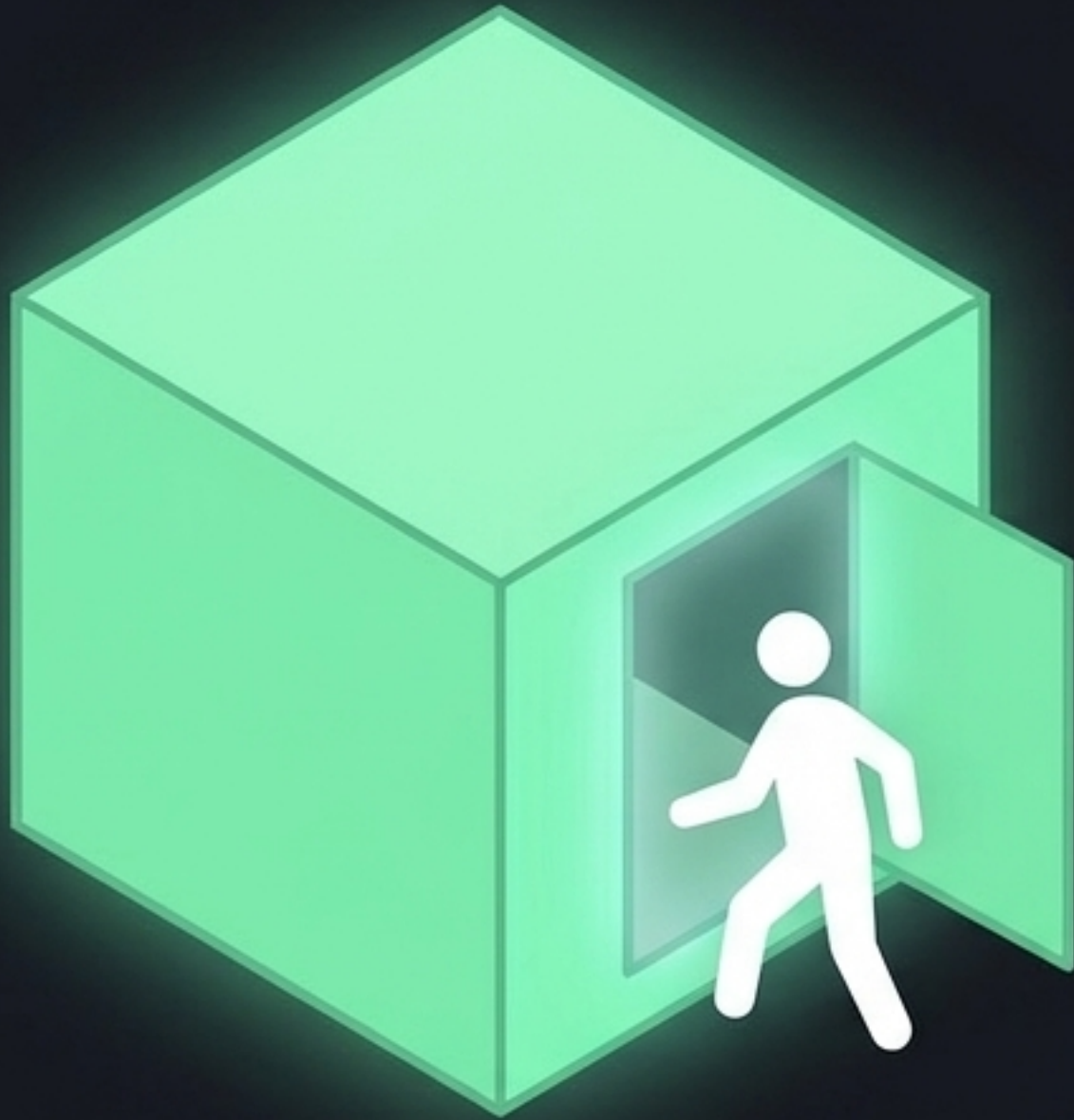
```
-q
```

returns only container IDs

The Container Lifecycle



Interacting with Containers

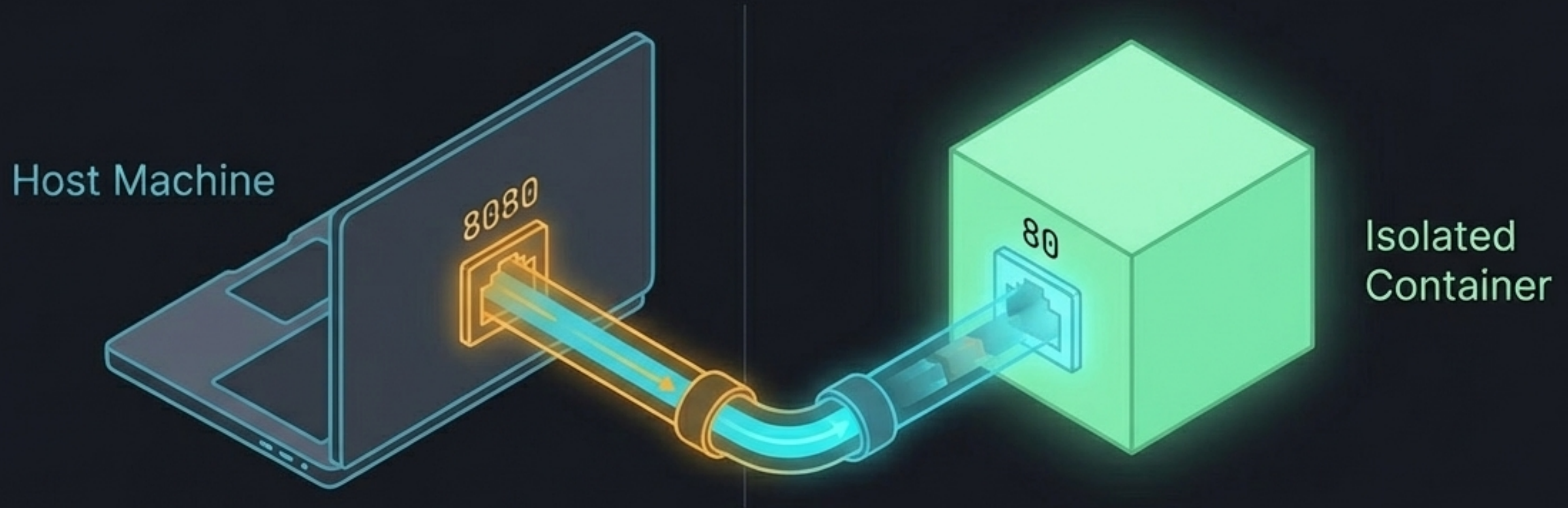


```
docker container exec -it  
<container_id> /bin/bash
```

Executes a secondary process inside an already-running container.

Unlike 'run' which spins up a brand new environment, 'exec' acts as a doorway into an existing, live ecosystem. The process dies when the container restarts.

System & Networking: Port Mapping



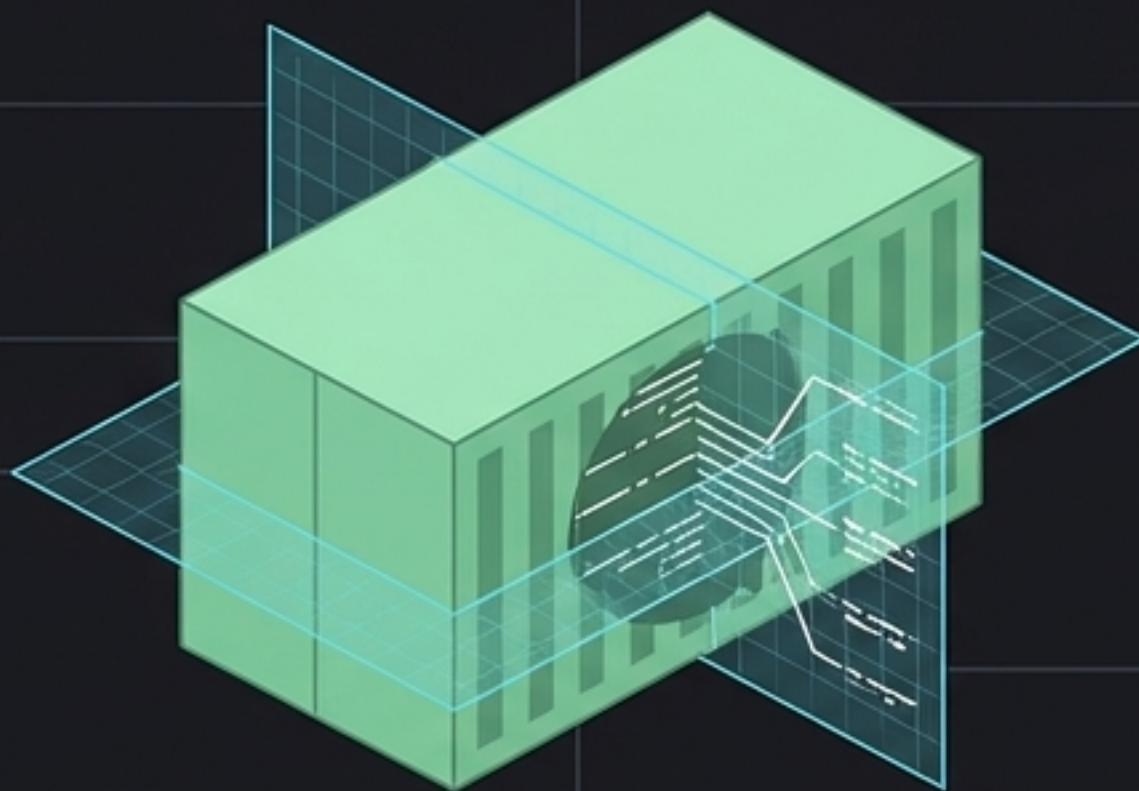
Command

```
docker run -p 8080:80 nginx
```

The Rule

Syntax is always `HOST_PORT : CONTAINER_PORT`. This conduit breaches the container's internal network isolation to allow outside traffic.

Diagnostic Tools & Deep Inspection



```
docker container top <id>
```

Displays active, running processes isolated inside that specific container. Like the Linux "top" command, but targeted.

...	
-	R19s
-	R19s
-	-
-	-
-	-

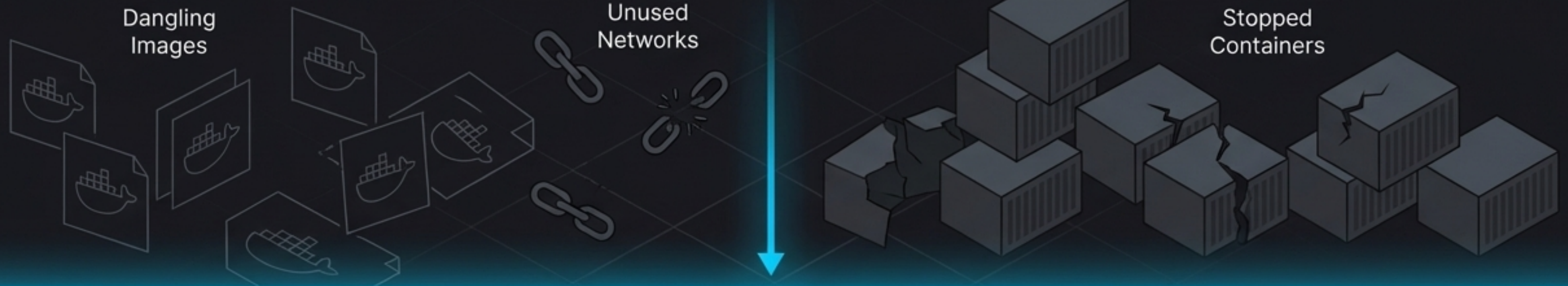
```
docker inspect <object_id>
```

Returns an exhaustive JSON array of low-level configuration data. Crucial for debugging network IPs, mounted volumes, and build errors.

```
{  
  "key": {},  
  "key": {...},  
  "key": {}  
}
```

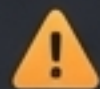

System Maintenance: Pruning

Before



`docker system prune`

After



A powerful, single-stroke command that permanently destroys all stopped containers, build cache, unused networks, and untagged dangling images.

Anatomy of an Execution

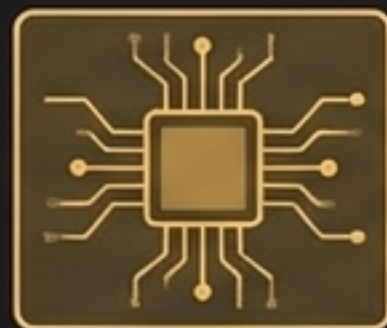
```
docker run -d -p 80:80 nginx
```

① Client Trigger



The CLI parses the string and sends a REST API request to the Daemon.

② Daemon Check



HOST

The Host checks for the nginx Image (Blueprint) locally.

④ Runtime Hand-off



Daemon passes the Image to containerd and runc.

⑤ Execution & Mapping



HOST
PORT 80
CONTAINER
PORT 80

runc isolates namespaces. The container spins up, and the -p flag maps Host Port 80 to Container Port 80.

③ Registry Pull

Missing locally, the Daemon pulls the Image from Docker Hub.



DOCKER HUB

End-to-End Orchestration complete.